Step 11 — Environment Variable Validation

logistiq · backend hardening, before deployment

> **What this fixes:** right now, four files read `process.env.DATABASE_URL`, `process.env.SESSION_SECRET`, and `process.env.NODE_ENV` directly, with no check that they're actually set or valid. If one is missing or malformed, the app doesn't fail until the exact line that happens to need it — which could be deep into a request, in production, instead of at startup. This step adds one file that checks everything up front, once, so a bad config fails loudly and immediately instead of quietly and later.

Before you start

I audited the codebase for every `process.env` read. There are exactly two real secrets: `DATABASE_URL` and `SESSION_SECRET`. Two other spots check `NODE_ENV`, which Next.js sets automatically — no risk there, but we'll route it through the same validated object for consistency.

> **Already done, with your approval:** your `SESSION_SECRET` was only 21 characters — short for a key that signs session JWTs (32+ random bytes is the standard; a short one is brute-forceable). I generated a proper 32-byte secret and it's already in your `.env`. One consequence: anyone currently signed in got logged out when that changed, since their token was signed with the old value. That's expected, not a bug.

What "validate at startup" actually means

Think of your env vars as ingredients a recipe assumes are already in the pantry. Right now, nothing checks the pantry before cooking starts — each recipe step just reaches for what it needs and assumes it's there. If something's missing, you don't find out until the exact step that needed it, which might be well into serving a customer instead of before you open the doors.

The fix below is a single file that checks the whole pantry once, the moment the app starts, using a schema (via `zod`, already in your `package.json`) that describes exactly what each variable should look like. Every other file then reaches into that already-checked object instead of the raw, unchecked `process.env`.

1 Create `app/lib/env.ts`

New file. This is the schema — the one place that knows what a valid config looks like.

app/lib/env.ts

```
/**
 * Validates every environment variable this app depends on, once, the first
 * time this module is imported — so a missing or malformed value fails loudly
 * at startup instead of wherever it happens to first get read at runtime.
 *
 * Everywhere else in the app should import `env` from this file instead of
 * reading `process.env` directly.
 */

import { z } from "zod";

const envSchema = z.object({
    DATABASE_URL: z
        .string({ message: "DATABASE_URL is not set — check your .env file" })
        .url({ message: "DATABASE_URL must be a valid connection string" }),

    SESSION_SECRET: z
        .string({ message: "SESSION_SECRET is not set — check your .env file" })
        .min(32, "SESSION_SECRET must be at least 32 characters (it signs
session JWTs — a short secret is brute-forceable)"),

    NODE_ENV: z.enum(["development", "production",
"test"]).default("development"),
});

const parsed = envSchema.safeParse(process.env);

if (!parsed.success) {
    console.error("Invalid environment variables:");
    console.error(parsed.error.flatten().fieldErrors);
    throw new Error("Invalid environment variables — see errors above, and check
them against app/lib/env.ts.");
}

export const env = parsed.data;
```

2 Update `app/lib/prisma.ts`

This file previously built the connection string as `` `${process.env.DATABASE_URL}` `` — a template literal, which is worth noticing on its own: if `DATABASE_URL` were ever unset, that line wouldn't throw, it

would silently produce the literal *string* `"undefined"`, and Prisma would fail later with a confusing connection error instead of a clear one. Swap it for the validated value.

app/lib/prisma.ts — replace the whole file with:

```
import "dotenv/config";
import { PrismaPg } from "@prisma/adapter-pg";
import { PrismaClient } from "@/generated/prisma/client";
import { env } from "./env";

const adapter = new PrismaPg({ connectionString: env.DATABASE_URL });
const prisma = new PrismaClient({ adapter });

export { prisma };
```

3 Update `app/lib/jwt.ts`

This file had its own one-off check for `SESSION_SECRET` — a separate safety net that duplicated what `env.ts` now does centrally. Remove it and use the validated value instead.

app/lib/jwt.ts — replace the whole file with:

```
/**
 * This file manages user authentication sessions by creating, verifying, and
deleting session tokens. It uses JWTs to securely store a session ID in the
user's browser while keeping the actual session information in the database with
Prisma.
 * On every request, it verifies the token, checks that the session still exists
and hasn't expired, and only considers the user authenticated if those checks
pass.
 */

import jwt from "jsonwebtoken"
import { env } from "./env";

export function signInSessionToken(sessionId: string) {
    return jwt.sign({ sid: sessionId }, env.SESSION_SECRET, { expiresIn: "30d"})
}

export function verifySessionToken(token: string): { sid: string } | null {
    try {
        const decoded = jwt.verify(token, env.SESSION_SECRET) as { sid:
string };
        return decoded;
    } catch {
        return null;
```

```
    }
}

// Session CRUD (create/read/delete) lives in ./session — kept here previously
as a
// duplicate, which risked the two copies drifting out of sync.
```

4 Update `app/lib/registry.ts`

Two small edits: add the import, and swap the one line that reads `process.env.NODE_ENV` directly.

app/lib/registry.ts — add this import, right after the existing USERROLE import:

```
import { env } from "./env";
```

app/lib/registry.ts — then change this line:

```
    if (REGISTRY.has(name) && process.env.NODE_ENV === "production") {
```

to:

```
    if (REGISTRY.has(name) && env.NODE_ENV === "production") {
```

5 Update `app/api/auth/sign-in/route.ts`

Same pattern: add the import, swap the one `process.env.NODE_ENV` read in the cookie config.

app/api/auth/sign-in/route.ts — add this import, right after the existing signInSessionToken import:

```
import { env } from "@/app/lib/env";
```

app/api/auth/sign-in/route.ts — then change this line (inside cookieStore.set):

```
            secure: process.env.NODE_ENV === "production",
```

to:

```
            secure: env.NODE_ENV === "production",
```

6 Verify

Run these, in order:

```
npx tsc --noEmit
npm run build
npx vitest run
```

`tsc` should show no new errors. `build` should complete normally. `vitest` should still show all tests passing, same as before this change — nothing here touches test logic, only how config is read.

> **I verified this first, so it isn't guesswork:** I applied all five edits above to a throwaway copy of your project (not your real files) and ran them there. Two things worth knowing:
>
> - `tsc --noEmit` on the edited copy produced zero new errors — the only two errors it shows (in `types/validator.ts`, about missing generated Next.js page/layout types) are pre-existing and appear identically on your untouched project; they only resolve after a successful `next build` generates those types, and have nothing to do with this change.
> - I couldn't run your actual `next build` or `vitest` in this environment — it's a different CPU architecture than your Mac, so Next's and esbuild's native binaries fail to load here, the same issue you'd have hit if you'd tried running your build in a totally different environment. So I tested the schema's actual logic directly instead: fed it your real values (passed), a missing secret (failed, correct message), your old 21-character secret (failed, correct message), and a malformed `DATABASE_URL` (failed, correct message). All four behaved exactly as designed.
>
> You're the first to run the full toolchain on your own machine — flag anything that doesn't match what's described here.

What's next

This closes out env validation. Two things still open from here:

- **Deployment (Step 11b)** — I checked your Vercel account: no project exists yet for logistiq (there are unrelated projects from other work — `swift-shift` variants, `okatsu` — but nothing for this app). Happy to write that guide next.
- **WarehouseFilter wiring** — the dashboard dropdown that doesn't actually filter anything yet. Still waiting whenever you want it.
